LAPORAN PRAKTIKUM

STRUKTUR DATA

ABSTRACT DATA TYPE (JAM)

PEKAN 1

Disusun Oleh :

KINAYA NOVRYA MANDA

(2511531016)

Dosen Pengampu :

DR. WAHYUDI, S.T, M.T

Asisten Praktikum :

MUHAMMAD GALID AVERO

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis sampaikan kehadirat Allah SWT. Salawat dan salam disampaikan kepada Nabi Muhammad SAW. Karena thaufik dan hidayah-Nya, laporan praktikum Java ini dapat diselesaikan dengan baik dan tepat waktu. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban dari kegiatan praktikum yang telah dilaksanakan, sekaligus sebagai sarana untuk memperdalam pemahaman mengenai konsep dasar serta penerapan bahasa pemrograman Java.

Melalui praktikum ini, penulis memperoleh pengalaman langsung dalam memahami Implementasi Abstact Data Type menggunakan Java. Diharapkan laporan ini dapat memberikan gambaran yang jelas mengenai materi yang telah dipelajari serta hasil dari percobaan yang dilakukan selama praktikum berlangsung.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna, baik dari segi isi maupun penyajiannya. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan di masa yang akan datang.

Akhir kata, penulis mengucapkan terima kasih kepada dosen pengampu, asisten laboratorium, serta semua pihak yang telah memberikan bimbingan, arahan, dan dukungan sehingga laporan praktikum ini dapat tersusun. Semoga laporan ini dapat bermanfaat bagi pembaca, khususnya dalam memahami dasar-dasar pemrograman Java.

Padang, April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....i

DAFTAR ISI.....ii

DAFTAR LAMPIRAN.....iii

BAB 1 PENDAHULUAN

1.1 Latar Belakang..... 1

1.2 Tujuan Praktikum..... 1

1.3 Manfaat Praktikum.....2

BAB 2 PEMBAHASAN

2.1 Kode Program..... 3

2.2 Langkah Kerja.....5

2.3 Analisis Hasil..... 13

BAB 3 KESIMPULAN

3.1 Hasil Praktikum.....14

3.2 Saran Pengembangan..... 16

DAFTAR KEPUSTAKAAN.....17

TUGAS PRAKTIKUM PEKAN 1.....18

DAFTAR LAMPIRAN

Kode Program 2.1 (Kode Program Jam).....3

Kode Program 2.2 (Kode Program JamDriver)..... 4

Kode Program 2.3 (Kode Program JamDriver2)..... 4

Gambar 2.1 (Langkah 1 Jam).....5

Gambar 2.2 (Langkah 2 Jam).....5

Gambar 2.3 (Langkah 3 Jam).....6

Gambar 2.4 (Langkah 4 Jam).....6

Gambar 2.5 (Langkah 5 Jam).....6

Gambar 2.6 (Langkah 6 Jam).....7

Gambar 2.7 (Langkah 7 Jam).....7

Gambar 2.8 (Langkah 1 JamDriver)..... 7

Gambar 2.9 (Langkah 2 JamDriver)..... 8

Gambar 2.10 (Langkah 3 JamDriver)..... 8

Gambar 2.11 (Langkah 4 JamDriver)..... 8

Gambar 2.12 (Langkah 5 JamDriver)..... 8

Gambar 2.13 (Langkah 6 JamDriver)..... 9

Gambar 2.14 (Langkah 7 JamDriver)..... 9

Gambar 2.15 (Langkah 8 JamDriver)..... 9

Gambar 2.16 (Langkah 1 JamDriver2)..... 10

Gambar 2.17 (Langkah 2 JamDriver2)..... 10

Gambar 2.18 (Langkah 3 JamDriver2)..... 10

Gambar 2.19 (Langkah 4 JamDriver2)..... 11

Gambar 2.20 (Langkah 5 JamDriver2)..... 11

Gambar 2.21 (Langkah 6 JamDriver2)..... 12

Gambar 2.22 (Langkah 7 JamDriver2)..... 12

Gambar 2.23 (Langkah 8 JamDriver2)..... 12

Gambar 2.24 (Langkah 9 JamDriver2)..... 12

Gambar 3.1 (Hasil Praktikum JamDriver)..... 14

Gambar 3.2 (Hasil Praktikum JamDriver2)..... 15

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman berorientasi objek, representasi data yang sederhana seperti waktu ternyata membutuhkan pengelolaan yang terstruktur agar dapat digunakan secara fleksibel dalam berbagai operasi. Waktu tidak hanya ditampilkan, tetapi juga perlu dihitung, dibandingkan, dan dimodifikasi sesuai kebutuhan. Oleh karena itu, pembuatan class yang merepresentasikan waktu menjadi penting sebagai latihan untuk memahami konsep dasar seperti atribut, method, serta interaksi antar objek dalam sebuah program.

Melalui praktikum ini, digunakan bahasa pemrograman Java untuk membangun class yang mampu menangani berbagai operasi waktu, seperti konversi ke detik, penjumlahan, pengurangan, hingga perbandingan dua objek waktu. Selain itu, penggunaan class driver juga membantu dalam menguji fungsi-fungsi yang telah dibuat dengan melibatkan input dari pengguna. Dengan demikian, praktikum ini tidak hanya berfokus pada penulisan kode, tetapi juga pada pemahaman alur logika program secara menyeluruh.

1.2 Tujuan Praktikum

Tujuan dilakukannya praktikum ini adalah sebagai berikut :

1. Memahami konsep dasar pemrograman berorientasi objek dalam Java.
2. Membuat dan mengelola class untuk merepresentasikan data waktu (jam, menit, detik).
3. Mengimplementasikan method untuk operasi waktu seperti konversi, penjumlahan, pengurangan, dan perbandingan.

4. Menggunakan input dari pengguna untuk menguji program melalui class driver.

1.3 Manfaat Praktikum

Manfaat dilakukannya praktikum ini adalah sebagai berikut :

1. Mahasiswa dapat memahami cara kerja class dan object dalam pemrograman Java.
2. Mahasiswa dapat mengolah data waktu menggunakan berbagai operasi secara sistematis.
3. Mahasiswa dapat melatih logika pemrograman dalam menyelesaikan permasalahan sederhana.
4. Mahasiswa memperoleh pengalaman dalam mengembangkan dan menguji program secara langsung.

BAB II

PEMBAHASAN

2.1 Kode Program

2.1.1 Jam_2511531016

```java
package pekan1_2511531016;

public final class Jam_2511531016 {
    private int hh; // 0..23
    private int mm; // 0..59
    private int ss; // 0..59

    // --- Validator ---
    public static boolean isValid (int h, int m, int s) {
        return (0 <= h && h <= 23) && (0 <= m && m <= 59) && (0 <= s && s <= 59);
    }

    // --- Konstruktor/Kreator ---
    public Jam_2511531016(int h, int m, int s) {
        this.hh = h; this.mm = m; this.ss = s;
    }

    // --- Selektor ---
    public int getHour() { return hh; }
    public int getMinute() { return mm; }
    public int getSecond() { return ss; }

    // --- Mutator ---
    public void setHour(int h) { this.hh = h; }
    public void setMinute(int m) { this.mm = m; }
    public void setSecond(int s) { this.ss = s; }

    // --- Konversi ---
    public int toSecond() { return hh * 3600 + mm * 60 + ss; }
    public static Jam_2511531016 fromSeconds(int total) {
        if (total < 0) throw new IllegalArgumentException("detik negatif");
        total %=24 * 3600; // wrap 24 jam
        int h = total / 3600; total %=3600;
        int m = total / 60; int s = total % 60;
        return new Jam_2511531016(h,m,s);
```

```java
    }

    // --- Relasional ---
    public int compareTo(Jam_2511531016 other) { return Integer.compare(this.toSecond(), other.toSecond()); }
    public boolean equals(Object o) {
        if (!(o instanceof Jam_2511531016 j)) return false;
        return hh == j.hh && mm == j.mm && ss == j.ss;
    }
    public int hashCode() { return java.util.Objects.hash(hh, mm, ss); }

    // --- Aritmatika ---
    public Jam_2511531016 plus(Jam_2511531016 other) { return fromSeconds(this.toSecond() + other.toSecond()); }
    public Jam_2511531016 minus(Jam_2511531016 other) { return fromSeconds(Math.floorMod(this.toSecond() - other.toSecond(), 24 * 3600)); }
    public Jam_2511531016 nextSecond() { return fromSeconds(this.toSecond() + 1); }
    public Jam_2511531016 nextNSeconds(int n) { return fromSeconds(this.toSecond() + Math.max(0, n)); }
    public Jam_2511531016 prevSecond() { return fromSeconds(Math.floorMod(this.toSecond() - 1, 24 * 3600)); }
    public Jam_2511531016 prevNSeconds(int n) { return fromSeconds(Math.floorMod(this.toSecond() - Math.max(0, n), 24 * 3600)); }

    // --- Durasi (detik) --- bisa negatif jika this > other? sesuai spesifikasi durasi (JAw, JAkh)
    public static int durasiDetik(Jam_2511531016 jaw, Jam_2511531016 jakh) { return jakh.toSecond() - jaw.toSecond(); }

    //overriding ke string
    public String toString() { return String.format("%02d:%02d:%02d", hh, mm, ss); }

}
```

Kode Program 2.1 (Kode Program Jam)

2.1.2 JamDriver

```java
package pekan1_2511531016;

public class JamDriver_2511531016 {
    public static void main(String[] args) {
        Jam_2511531016 a = new Jam_2511531016(23, 59, 50);
        Jam_2511531016 b = new Jam_2511531016(0, 0, 15);
        System.out.println("a        = " + a);
        System.out.println("b        = " + b);
        System.out.println("a+b      = " + a.plus(b));
        System.out.println("next 20s = " + a.nextNSeconds(20));
        System.out.println("durasi(a,b) = " + Jam_2511531016.durasiDetik(a, b));
        System.out.println("a.compareTo(b) = " + a.compareTo(b));
    }
}
```

Kode Program 2.2 (Kode Program JamDriver)

2.1.3 JamDrive2

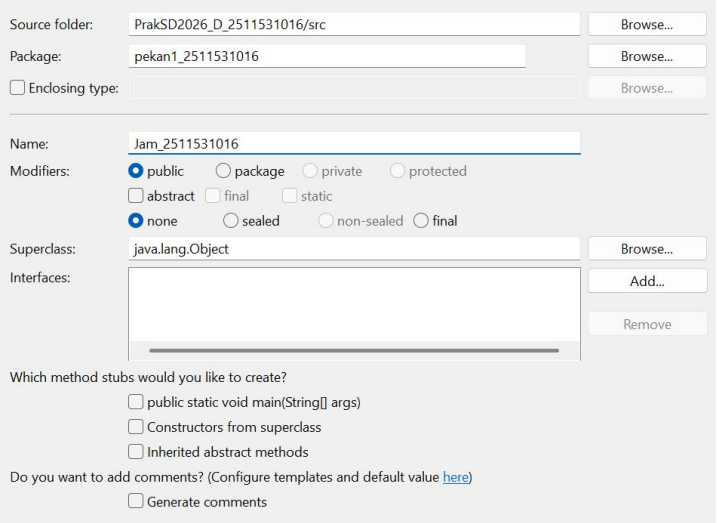
```java
package pekan1_2511531016;
import java.util.Scanner;

public class JamDriver2_2511531016 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.println("=== Program Driver Objek Jam ===");
        // 1. Input Jam Pertama
        System.out.println("\n--- Input Jam 1 ---");
        Jam_2511531016 j1 = buatJamDariInput(input);

        // 2. Input Jam Kedua
        System.out.println("\n--- Input Jam 2 ---");
        Jam_2511531016 j2 = buatJamDariInput(input);

        // 3. Menampilkan Data
        System.out.println("\n--- Hasil Operasi ---");
        System.out.println("Jam 1 (String) : " + j1.toString());
        System.out.println("Jam 2 (String) : " + j2.toString());
        System.out.println("Jam 1 dalam detik : " + j1.toSecond());
        System.out.println("Jam 2 dalam detik : " + j2.toSecond());

        //4. Operasi Relasional (Perbandfingan)
        int perbandingan = j1.compareTo(j2);
        if (perbandingan > 0) {
            System.out.println("Status : Jam 1 lebih lambat (setelah) Jam 2");
        } else if (perbandingan < 0) {
            System.out.println("Status : Jam 1 lebih l awal (sebelum) Jam 2");
        } else {
            System.out.println("Status : Jam 1 dan Jam 2 sama persis");
        }

        // 5. Operasi Aritmatika
        System.out.println("Durasi (j1 ke j2) : " + Jam_2511531016.durasiDetik(j1, j2) + " detik");

        Jam_2511531016 jNext = j1.nextSecond();
        System.out.println("Jam 1 Detik Berikutnya: " + jNext);

        Jam_2511531016 jPrev = j1.prevSecond();
        System.out.println("Jam 1 Detik Sebelumnya: " + jPrev);

        //6. Operasi Pennjumlahan Jam
        Jam_2511531016 jHasilPlus = j1.plus(j2);
        System.out.println("Hasil j1 + J2 : " + jHasilPlus);

        input.close();
        System.out.println("\nProgram Selesai.");
    }

    private static Jam_2511531016 buatJamDariInput(Scanner sc) {
        int h, m, s;
        while (true) {
            System.out.print("Masukkan Jam (0-23) : ");
            h = sc.nextInt();
            System.out.print("Masukkan Menit (0-59) : ");
            m = sc.nextInt();
            System.out.print("Masukkan Detik (0-59) : ");
            s = sc.nextInt();

            // Memanggil method static isValid dari kelas Jam
            if (Jam_2511531016.isValid(h,  m,  s)) {
                return new Jam_2511531016(h, m, s);
            } else {
                System.out.println("[Error] Input tidak valid! Silahkan ulangi.\n");
            }
        }
    }
}
```

Kode Program 2.3 (Kode Program JamDriver2)

2.2 Langkah Kerja

2.2.1 Jam

1. Membuat kelas baru bernama *Jam_2511531016* dalam *package Pekan1_2511531016.*



Gambar 2.1 (Langkah 1 Jam)

2. Buat final class Jam_2511531016 yang berfungsi untuk merepresentasikan waktu dalam bentuk jam, menit, dan detik. Pada bagian ini juga ditentukan atribut hh, mm, ss serta method validasi untuk memastikan nilai yang dimasukkan sesuai dengan batas yang diperbolehkan.

```
public final class Jam_2511531016 {
    private int hh; // 0..23
    private int mm; // 0..59
    private int ss; // 0..59

    // --- Validator ---
    public static boolean isValid (int h, int m, int s) {
        return (0 <= h && h <= 23) && (0 <= m && m <= 59) && (0 <= s && s <= 59);
    }
```

Gambar 2.2 (Langkah 2 Jam)

3. Selanjutnya, buat konstruktor untuk menginisialisasi nilai jam, menit, dan detik, serta method setter & getter untuk mengakses dan mengubah nilai tersebut.

```
// --- Konstruktor/Kreator ---
public Jam_2511531016(int h, int m, int s) {
this.hh = h; this.mm = m; this.ss = s;
}
// --- Selektor ---
public int getHour() { return hh; }
public int getMinute() { return mm; }
public int getSecond() { return ss; }
// --- Mutator ---
public void setHour(int h) { this.hh = h; }
public void setMinute(int m) { this.mm = m; }
public void setSecond(int s) { this.ss = s; }
```

Gambar 2.3 (Langkah 3 Jam)

4. Setelah itu, buat fungsi konversi waktu ke dalam satuan detik dan sebaliknya untuk mempermudah proses perhitungan waktu.

```
// --- Konversi ---
public int toSecond() { return hh * 3600 + mm * 60 + ss; }
public static Jam_2511531016 fromSeconds(int total) {
    if (total < 0) throw new IllegalArgumentException("detik negatif");
    total %=24 * 3600; // wrap 24 jam
    int h = total / 3600; total %=3600;
    int m = total / 60; int s = total % 60;
    return new Jam_2511531016(h,m,s);
}
```

Gambar 2.4 (Langkah 4 Jam)

5. Buat juga fungsi relasional untuk membandingkan dua objek waktu, baik untuk mengetahui urutan maupun kesamaan nilai.

```
// --- Relasional ---
public int compareTo(Jam_2511531016 other) { return Integer.compare(this.toSecond(), other.toSecond()); }
public boolean equals(Object o) {
    if (!(o instanceof Jam_2511531016 j)) return false;
    return hh == j.hh && mm == j.mm && ss == j.ss;
}
public int hashCode() { return java.util.Objects.hash(hh, mm, ss); }
```

Gambar 2.5 (Langkah 5 Jam)

6. Tambahkan operasi aritmatika pada waktu seperti penjumlahan, pengurangan, serta penambahan atau pengurangan detik secara bertahap.

```java
// --- Aritmatika ---
public Jam_2511531016 plus(Jam_2511531016 other) { return fromSeconds(this.toSecond() + other.toSecond()); }
public Jam_2511531016 minus(Jam_2511531016 other) { return fromSeconds(Math.floorMod(this.toSecond() - other.toSecond(), 24 * 3600)); }
public Jam_2511531016 nextSecond() { return fromSeconds(this.toSecond() + 1); }
public Jam_2511531016 nextNSeconds(int n) { return fromSeconds(this.toSecond() + Math.max(0, n)); }
public Jam_2511531016 prevSecond() { return fromSeconds(Math.floorMod(this.toSecond() - 1, 24 * 3600)); }
public Jam_2511531016 prevNSeconds(int n) { return fromSeconds(Math.floorMod(this.toSecond() - Math.max(0, n), 24 * 3600)); }
```

Gambar 2.6 (Langkah 6 Jam)

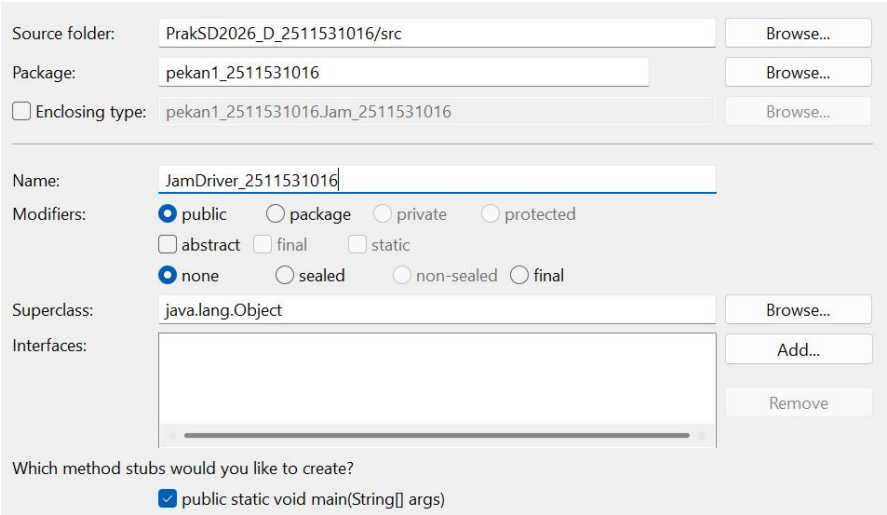
7. Terakhir, buatlah fungsi untuk menghitung durasi antar dua waktu serta menampilkan waktu dalam format yang rapi agar mudah dibaca.

```java
// --- Durasi (detik) --- bisa negatif jika this > other? sesuai spesifikasi durasi (JAw, JAkh)
public static int durasiDetik(Jam_2511531016 jaw, Jam_2511531016 jakh) { return jakh.toSecond() - jaw.toSecond(); }

//overriding ke string
public String toString() { return String.format("%02d:%02d:%02d", hh, mm, ss); }
```

Gambar 2.7 (Langkah 7 Jam)

2.2.2 JamDriver

1. Membuat kelas baru bernama *JamDriver_2511531016* dalam *package Pekan1_2511531016.*



Gambar 2.8 (Langkah 1 JamDriver)

2. Program dijalankan dan masuk ke *main.*

```
public static void main(String[] args) {
```

Gambar 2.9 (Langkah 2 JamDriver)

3. Buat dua objek waktu, yaitu a dan b, dengan nilai jam, menit, dan detik yang berbeda sebagai data awal untuk diuji.

```
Jam_2511531016 a = new Jam_2511531016(23, 59, 50);
Jam_2511531016 b = new Jam_2511531016(0, 0, 15);
```

Gambar 2.10 (Langkah 3 JamDriver)

4. Setelah objek dibuat, program menampilkan nilai dari masing-masing objek menggunakan system.out.println. Pada bagian ini, method *toString()* dari *class* sebelumnya akan otomatis dipanggil untuk menampilkan format waktu yaitu hh, mm, ss.

```
System.out.println("a = " + a);
System.out.println("b = " + b);
```

Gambar 2.11 (Langkah 4 JamDriver)

5. Selanjutnya lakukan operasi penjumlahan waktu antara objek a dan b menggunakan *method plus()*. Hasilnya langsung ditampilkan ke layar.

```
System.out.println("a+b = " + a.plus(b));
```

Gambar 2.12 (Langkah 5 JamDriver)

6. Lakukan penambahan waktu sebesar 20 detik dari projek a menggunakan *method nextNSeconds().*

```
System.out.println("next 20s = " + a.nextNSeconds(20));
```

Gambar 2.13 (Langkah 6 JamDriver)

7. Kemudian, hitung selisih atau durasi antara waktu a dan b dalam satuan detik menggunakan *method statis durasiDetik().*

```
System.out.println("durasi(a,b) = " + Jam_2511531016.durasiDetik(a, b));
```

Gambar 2.14 (Langkah 7 JamDriver)

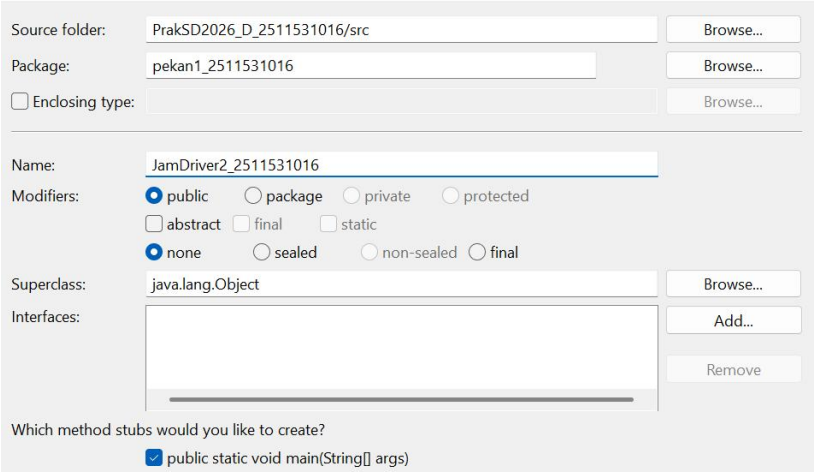
8. Setelah itu, bandingkan dua objek waktu a dan b menggunakan *method compareTo()* untuk mengetahui hubungan nilainya (lebih besar, sama, atau lebih kecil).

```
System.out.println("a.compareTo(b) = " + a.compareTo(b));
```

Gambar 2.15 (Langkah 8 JamDriver)

2.2.3 JamDriver2

1. Membuat kelas baru bernama *JamDriver2_2511531016* dalam *package Pekan1_2511531016.*



Gambar 2.16 (Langkah 1 JamDriver2)

2. Program dimulai dari method main dengan membuat objek Scanner untuk membaca input dari pengguna. Pada tahap ini juga ditampilkan judul program sebagai penanda awal proses.

```
public static void main(String[] args) {
Scanner input = new Scanner(System.in);
System.out.println("=== Program Driver Objek Jam ===");
```

Gambar 2.17 (Langkah 2 JamDriver2)

3. Selanjutnya, program meminta pengguna untuk menginput data waktu pertama (Jam 1) dan kedua (Jam 2). Lalu, simpan ke objek j1 dan j2.

```
// 1. Input Jam Pertama
System.out.println("\n--- Input Jam 1 ---");
Jam_2511531016 j1 = buatJamDariInput(input);
// 2. Input Jam Kedua
System.out.println("\n--- Input Jam 2 ---");
Jam_2511531016 j2 = buatJamDariInput(input);
```

Gambar 2.18 (Langkah 3 JamDriver2)

4. Kemudian tampilkan hasil input kedua objek dalam bentuk string serta dalam bentuk detik menggunakan *method toString()* dan *toSecond()*.

```
// 3. Menampilkan Data
System.out.println("\n--- Hasil Operasi ---");
System.out.println("Jam 1 (String) : " + j1.toString());
System.out.println("Jam 2 (String) : " + j2.toString());
System.out.println("Jam 1 dalam detik : " + j1.toSecond());
System.out.println("Jam 2 dalam detik : " + j2.toSecond());
```

Gambar 2.19 (Langkah 4 JamDriver2)

5. Lakukan operasi relasional dengan membandingkan kedua objek waktu menggunakan *method compareTo()*. Berdasarkan nilai perbandingan, program menentukan kondisi apakah Jam 1 lebih lambat/awal/sama dengan Jam 2, lalu menampilkan hasilnya menggunakan percabangan *if-else.*

```
//4. Operasi Relasional (Perbandingan)
int perbandingan = j1.compareTo(j2);
if (perbandingan > 0) {
System.out.println("Status : Jam 1 lebih lambat (setelah) Jam 2");
} else if (perbandingan < 0) {
System.out.println("Status : Jam 1 lebih l awal (sebelum) Jam 2");
} else {
System.out.println("Status : Jam 1 dan Jam 2 sama persis");
}
```

Gambar 2.20 (Langkah 5 JamDriver2)

6. Lakukan operasi aritmatika waktu dengan menghitung durasi antara j1 dan j2 dalam satuan detik menggunakan method *durasiDetik()*. Tampilkan waktu satu detik setelah dan sebelum menggunakan *nextSeconnd()* dan *prevSecond()*.

```
// 5. Operasi Aritmatika
System.out.println("Durasi (j1 ke j2) : " +
Jam_2511531016.durasiDetik(j1, j2) + " detik");
Jam_2511531016 jNext = j1.nextSecond();
System.out.println("Jam 1 Detik Berikutnya: " + jNext);
Jam_2511531016 jPrev = j1.prevSecond();
System.out.println("Jam 1 Detik Sebelumnya: " + jPrev);
```

Gambar 2.21 (Langkah 6 JamDriver2)

7. Lakukan penjumlahan dua objek waktu j1 dan j2 menggunakan method plus(). Lalu, tutup input dan akhiri program.

```
//6. Operasi Penjumlahan Jam
Jam_2511531016 jHasilPlus = j1.plus(j2);
System.out.println("Hasil j1 + J2 : " + jHasilPlus);
input.close();
System.out.println("\nProgram Selesai.");
}
```

Gambar 2.22 (Langkah 7 JamDriver2)

8. Lakukan proses input berulang (loop) sampai nilai yang dimasukkan sesuai dengan rentang yang ditentukan.

```
private static Jam_2511531016 buatJamDariInput(Scanner sc) {
int h, m, s;
while (true) {
System.out.print("Masukkan Jam (0-23) : ");
h = sc.nextInt();
System.out.print("Masukkan Menit (0-59) : ");
m = sc.nextInt();
System.out.print("Masukkan Detik (0-59) : ");
s = sc.nextInt();
```

Gambar 2.23 (Langkah 8 JamDriver2)

9. Validasi input menggunakan method isValid. Jika valid objek jam adibuat dan dikembalikan, jika tidak maka akan ditampilkan pesan error dan input diulang.

```
// Memanggil method static isValid dari kelas Jam
if (Jam_2511531016.isValid(h, m, s)) {
return new Jam_2511531016(h, m, s);
} else {
System.out.println("[Error] Input tidak valid! Silahkan ulangi.\n");
}
```

Gambar 2.24 (Langkah 9 JamDriver2)

2.3 Analisis Hasil

Berdasarkan hasil praktikum, program yang dibuat sudah mampu merepresentasikan waktu dalam bentuk objek dengan cukup baik melalui class yang berisi atribut jam, menit, dan detik. Berbagai method yang disediakan, seperti konversi ke detik, penjumlahan, pengurangan, serta perbandingan waktu, dapat berjalan sesuai dengan logika yang diharapkan. Hal ini terlihat dari output yang dihasilkan, di mana waktu dapat ditampilkan dalam format yang rapi dan juga diolah dalam bentuk perhitungan tanpa mengalami kesalahan yang berarti.

Selain itu, penggunaan class driver dengan input dari pengguna membuat program terasa lebih interaktif dan tidak hanya bersifat statis. Proses membandingkan dua waktu menggunakan method compareTo juga menunjukkan bahwa objek dapat diperlakukan secara logis seperti data numerik. Dari sini terlihat bahwa konsep dasar pemrograman berorientasi objek, seperti enkapsulasi dan penggunaan method, sudah diterapkan dengan cukup baik. Namun, masih ada beberapa bagian yang bisa diperhatikan lebih lanjut, seperti validasi input agar program lebih aman saat menerima data dari pengguna.

BAB III
KESIMPULAN

3.1 Hasil Praktikum

3.1.1 JamDriver

```
Problems  @ Javadoc  Declaration  Console ×
<terminated> JamDriver_2511531016 [Java Application] C:\Users
a         = 23:59:50
b         = 00:00:15
a+b       = 00:00:05
next 20s = 00:00:10
durasi(a,b) = -86375
a.compareTo(b) = 1
```

Gambar 3.1 (Hasil Praktikum JamDriver)

Output tersebut menunjukkan bahwa waktu a berada di akhir hari (23:59:50) dan b di awal hari (00:00:15), sehingga saat dijumlahkan hasilnya melewati batas 24 jam dan kembali ke 00:00:05. Penambahan 20 detik dari a juga membuat waktu berpindah ke hari berikutnya menjadi 00:00:10. Nilai durasi yang negatif (-86375) menunjukkan bahwa b dianggap lebih awal dari a, sedangkan hasil *compareTo = 1* menandakan bahwa waktu a lebih besar atau lebih lambat dibandingkan b.

3.1.2 JamDriver2

```
Problems  @ Javadoc  Declaration  Console ×
<terminated> JamDriver2_2511531016 [Java Application] C:\Us
=== Program Driver Objek Jam ===

--- Input Jam 1 ---
Masukkan Jam (0-23) : 22
Masukkan Menit (0-59) : 59
Masukkan Detik (0-59) : 45

--- Input Jam 2 ---
Masukkan Jam (0-23) : 21
Masukkan Menit (0-59) : 2
Masukkan Detik (0-59) : 17

--- Hasil Operasi ---
Jam 1 (String) : 22:59:45
Jam 2 (String) : 21:02:17
Jam 1 dalam detik : 82785
Jam 2 dalam detik : 75737
Status : Jam 1 lebih lambat (setelah) Jam 2
Durasi (j1 ke j2) : -7048 detik
Jam 1 Detik Berikutnya: 22:59:46
Jam 1 Detik Sebelumnya: 22:59:44
Hasil j1 + J2 : 20:02:02

Program Selesai.
```

Gambar 3.2 (Hasil Praktikum JamDriver2)

Output tersebut menunjukkan bahwa Jam 1 (22:59:45) memiliki waktu yang lebih besar dibandingkan Jam 2 (21:02:17), sehingga statusnya ditampilkan sebagai lebih lambat (setelah). Saat dikonversi ke detik, terlihat perbedaan nilainya cukup jauh, dan hasil durasi yang negatif (-7048 detik) menandakan bahwa Jam 2 terjadi lebih awal dari Jam 1. Selain itu, program juga menampilkan perubahan waktu satu detik setelah dan sebelum Jam 1, serta hasil penjumlahan kedua waktu yang menghasilkan 20:02:02, yang menunjukkan adanya perhitungan waktu dengan konsep modulo 24 jam.

3.2 Saran Pengembangan

Untuk pengembangan selanjutnya, program dapat ditingkatkan dengan menambahkan validasi input yang lebih ketat, terutama untuk memastikan nilai jam, menit, dan detik tidak melebihi batas yang telah ditentukan. Selain itu, program juga bisa dikembangkan agar mampu menangani kondisi khusus, seperti perhitungan waktu yang melewati batas hari secara lebih fleksibel, sehingga hasilnya tetap akurat dalam berbagai situasi.

Di sisi lain, tampilan output program juga dapat dibuat lebih informatif, misalnya dengan menambahkan keterangan yang lebih jelas pada setiap hasil operasi agar mudah dipahami oleh pengguna. Pengembangan lain yang bisa dilakukan adalah menambahkan fitur tambahan, seperti konversi ke format waktu lain atau integrasi dengan sistem yang lebih kompleks. Dengan pengembangan tersebut, program tidak hanya berfungsi sebagai latihan dasar, tetapi juga bisa menjadi lebih aplikatif dan mendekati kebutuhan nyata.

DAFTAR PUSTAKA

[1] Kementerian Pendidikan dan Kebudayaan, *Modul Pembelajaran Pemrograman Berorientasi Objek*. Jakarta: Kemendikbud, 2020.

[2] A. Kadir, *Pemrograman Java untuk Pemula*. Yogyakarta: Andi, 2021.

[3] B. Raharjo, *Pemrograman Berorientasi Objek Menggunakan Java*. Bandung: Informatika, 2021.

[4] W. Komputer, *Belajar Pemrograman Java Modern*. Yogyakarta: Andi, 2022.

[5] R. Munir, *Algoritma dan Pemrograman dalam Bahasa Java*. Bandung: Informatika, 2020.

TUGAS PRAKTIKUM

PEKAN 1

1. Soal

Buatlah sebuah program ADT mobil yang memiliki atribut : nama, tahun, cc, harga, dan merk.

Contoh : Avanza, 2020, 1300, 150000000, Toyota

Buatlah method ADT mobil tersebut : tambah mobil, hapus mobil, selektor dam mutator.

Buatlah 2 kelas : 1 kelas ADT mobil dan 1 kelas driver. Setiap kelas lengkapi dengan NIM.

2. Kode Program

2.1 ADTMobil_2511531016

```java
package pekan1_2511531016;

public class ADTMobil_2511531016 {
    private String nama;
    private int tahun, cc;
    private double harga;
    private String merk;

    private static ADTMobil_2511531016[] data = new ADTMobil_2511531016[10];
    private static int jumlah = 0;

    // Konstruktor
    public ADTMobil_2511531016(String nama, int tahun, int cc, double harga, String merk) {
        this.nama = nama;
        this.tahun = tahun;
        this.cc = cc;
        this.harga = harga;
        this.merk = merk;
    }

    // SeleKtor
    public String getNama() { return nama; }
    public int getTahun() { return tahun; }
    public int getCc() { return cc; }
    public double getHarga() { return harga; }
    public String getMerk() { return merk; }

    // Mutator
    public void setNama(String nama) { this.nama = nama; }
    public void setTahun(int tahun) { this.tahun = tahun; }
    public void setCc(int cc) { this.cc = cc; }
    public void setHarga(double harga) { this.harga = harga; }
    public void setMerk(String merk) { this.merk = merk; }
```

```java
    // Tambah mobil
    public static void tambahMobil(ADTMobil_2511531016 m) {
        if (jumlah < data.length) {
            data[jumlah] = m;
            jumlah++;
        } else {
            System.out.println("Data penuh!");
        }
    }

    // Hapus mobil
    public static void hapusMobil(int index) {
        if (index >= 0 && index < jumlah) {
            for (int i = index; i < jumlah - 1; i++) {
                data[i] = data[i + 1];
            }
            data[jumlah - 1] = null;
            jumlah--;
        } else {
            System.out.println("Index tidak valid!");
        }
    }

    // Tampilkan
    public static void tampil() {
        for (int i = 0; i < jumlah; i++) {
            System.out.println(i + ". " + data[i]);
        }
    }

    public String toString() {
        return nama + " | " + tahun + " | " + cc + "cc | " + String.format("%.0f", harga) + " | " + merk;
    }
}
```

Penjelasan :

Pada class ADT Mobil, program digunakan untuk merepresentasikan data mobil yang terdiri dari atribut nama, tahun, cc, harga, dan merk. Atribut dibuat private agar data lebih terjaga, kemudian disediakan method getter *(selector)* dan setter *(mutator)* untuk mengakses dan mengubah nilai atribut secara terkontrol. Selain itu, pada class ini juga terdapat array dan variabel jumlah yang digunakan untuk menyimpan beberapa objek mobil. Method *tambahMobil()* berfungsi untuk menambahkan data ke dalam array, sedangkan *hapusMobil()* digunakan untuk menghapus data dengan cara menggeser elemen agar tidak terjadi kekosongan di tengah. Method *toString()* digunakan untuk menampilkan data mobil dalam format yang lebih rapi dan mudah dibaca.

2.2 MobilDriver_2511531016

```java
package pekan1_2511531016;
import java.util.Scanner;

public class MobilDriver_2511531016 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Masukkan jumlah mobil: ");
        int n = sc.nextInt();
        sc.nextLine();

        for (int i = 0; i < n; i++) {
            System.out.println("\nData Mobil ke-" + (i + 1));

            System.out.print("Nama: ");
            String nama = sc.nextLine();

            System.out.print("Tahun: ");
            int tahun = sc.nextInt();

            System.out.print("CC: ");
            int cc = sc.nextInt();

            System.out.print("Harga: ");
            double harga = sc.nextDouble();
            sc.nextLine();

            System.out.print("Merk: ");
            String merk = sc.nextLine();

            ADTMobil_2511531016.tambahMobil(
                new ADTMobil_2511531016(nama, tahun, cc, harga, merk)
            );
        }

        System.out.println("\n=== Data Mobil ===");
        ADTMobil_2511531016.tampil();

        System.out.print("\nHapus index ke: ");
        int idx = sc.nextInt();
        ADTMobil_2511531016.hapusMobil(idx);

        System.out.println("\n=== Setelah Hapus ===");
        ADTMobil_2511531016.tampil();

        sc.close();
    }
}
```

Penjelasan :

Class Mobil Driver berfungsi sebagai pengendali jalannya program. Pada method main, pengguna diminta untuk memasukkan data mobil, kemudian data tersebut ditambahkan ke dalam ADT melalui method *tambahMobil()*. Setelah semua data dimasukkan, program menampilkan seluruh data yang tersimpan. Selanjutnya, pengguna diminta untuk memilih index data yang akan dihapus, lalu program memanggil method *hapusMobil()* untuk menghapus data tersebut. Setelah proses penghapusan, data ditampilkan kembali untuk menunjukkan perubahan yang terjadi. Alur program ini cukup jelas, yaitu mulai dari input data, penyimpanan, penampilan, hingga penghapusan data.

3. Output Kode Program

```
Problems  @ Javadoc  Declaration  Console ×
<terminated> MobilDriver_2511531016 [Java Application] C:\User
Masukkan jumlah mobil: 3

Data Mobil ke-1
Nama: Mazda 3
Tahun: 2025
CC: 1998
Harga: 579000000
Merk: Mazda

Data Mobil ke-2
Nama: HRV
Tahun: 2023
CC: 1498
Harga: 440000000
Merk: Honda

Data Mobil ke-3
Nama: Raize
Tahun: 2024
CC: 1200
Harga: 300000000
Merk: Toyota

=== Data Mobil ===
0. Mazda 3 | 2025 | 1998cc | 579000000 | Mazda
1. HRV | 2023 | 1498cc | 440000000 | Honda
2. Raize | 2024 | 1200cc | 300000000 | Toyota

Hapus index ke: 2

=== Setelah Hapus ===
0. Mazda 3 | 2025 | 1998cc | 579000000 | Mazda
1. HRV | 2023 | 1498cc | 440000000 | Honda
```

Penjelasan :

Dari hasil output yang ditampilkan, program berjalan sesuai dengan alur yang dirancang. Awalnya pengguna memasukkan jumlah data mobil sebanyak 3, kemudian setiap data diisi satu per satu mulai dari nama, tahun, cc, harga, hingga merk. Setelah semua data dimasukkan, program menampilkan seluruh data mobil dalam bentuk daftar dengan format yang sudah dirapikan, sehingga mudah untuk dibaca dan dibandingkan. Terlihat bahwa setiap objek mobil berhasil disimpan dengan benar di dalam array dan dapat dipanggil kembali saat ditampilkan.

Selanjutnya, ketika pengguna memilih untuk menghapus data pada index ke-2, program memproses penghapusan dengan cara menggeser data yang ada, sehingga tidak ada posisi yang kosong di tengah array. Hasil akhirnya terlihat bahwa data mobil "Raize" sudah tidak ada, dan hanya tersisa dua data yaitu Mazda 3 dan HRV. Dari sini bisa dipahami bahwa fungsi tambah dan hapus pada ADT sudah berjalan dengan baik, serta interaksi antara class Driver dan class Mobil juga sudah sesuai dengan yang diharapkan.